

Creazione della password tramite invio email

Invito utente e reset password con magic-link basati su token (Tappa T4)

Ambito: attivazione account (primo accesso) e reset password via link monouso inviato per email.

Principio: nessuno — admin compreso — conosce mai la password di un utente: la sceglie l'utente dal link.

Stack: .NET 10 · Blazor (pagine static SSR + endpoint minimal-API) · Dapper · SQL Server · MailKit · PBKDF2.

Riferimento: mirror di ICMVerbalì · **Migration:** `013_UtenteToken.sql` · **Generato il:** 10 giugno 2026.

Indice

1. Visione d'insieme e obiettivi di sicurezza

2. I due flussi: invito e reset

Flusso di invito (attivazione)

Flusso di reset (password dimenticata)

3. Modello dati: `fatt.UtenteToken`

Schema e migration

Stato derivato dalle colonne

Macchina a stati del token

4. Generazione del token e hashing (perché si salva solo l'hash)

5. Repository: transazioni e sentinel TOCTOU

6. Manager: validazione, ordine dei motivi, consumo atomico

7. La doppia difesa contro le race condition

8. Policy della password condivisa

9. Invio email: `IEmailSender`, SMTP e modalità sviluppo

10. Endpoint di autenticazione e pagine

Gli endpoint minimal-API

Le pagine static-rendered

Rate limiting e risposta neutra

11. Allineamento della gestione utenti

12. Configurazione e segreti (Brevo)

13. Elenco esaustivo dei punti di codice toccati

14. Verifica e test

1. Visione d'insieme e obiettivi di sicurezza

L'obiettivo è che un utente **scelga da sé** la propria password, senza che questa transiti mai per l'amministratore o per il database in chiaro. L'admin **invita** creando l'utente senza password; l'utente riceve un'email con un **link monouso** e imposta la password. Lo stesso meccanismo serve il **reset** ("password dimenticata"). Gli obiettivi di sicurezza guidano ogni scelta implementativa:

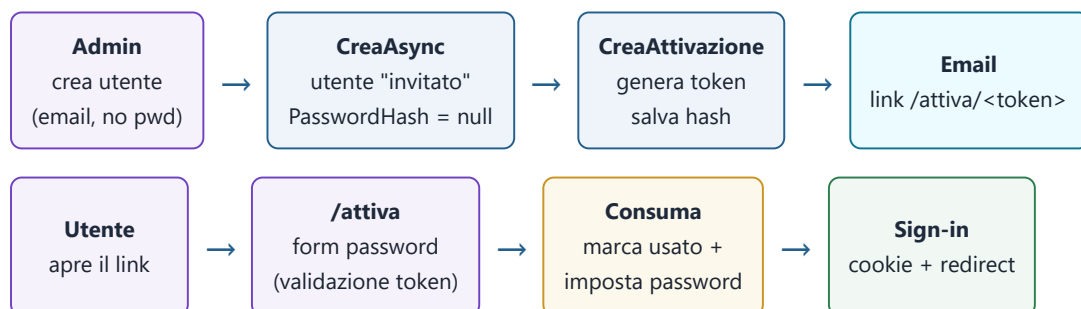
Obiettivo	Come è garantito
La password non è mai nota a terzi	L'admin non la imposta; non esiste UI per farlo
Il token nel DB non è riutilizzabile se rubato	Si salva solo l' hash SHA-256 , mai il token in chiaro
Uso singolo del link	Colonna UsatoUtc + sentinel transazionale
Validità limitata nel tempo	Colonna ScadenzaUtc (7 giorni invito, 1 ora reset)
Un reinvio invalida i link precedenti	Colonna RevocatoUtc impostata atomicamente
Niente enumerazione degli account	"Password dimenticata" risponde sempre allo stesso modo
Niente brute-force/spam sul reset	Rate limiting per IP sull'endpoint forgot-password
Password robusta	Policy condivisa (min 10 + maiuscola/minuscola/cifra)

Una sola differenza concettuale tra invito e reset

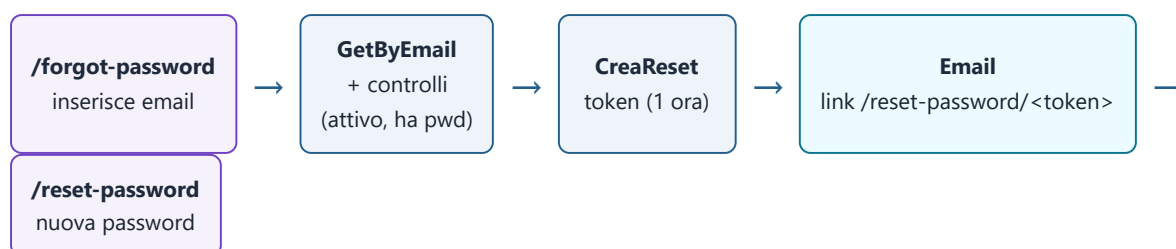
È il **tipo** di token (**Attivazione** vs **Reset**) e la sua durata. Il resto del meccanismo — generazione, hashing, validazione, consumo atomico, sign-in finale — è identico e condiviso.

2. I due flussi: invito e reset

2.1 Flusso di invito (attivazione)



2.2 Flusso di reset (password dimenticata)



Da notare due asimmetrie volute: il reset è offerto solo a utenti **attivi e già attivati** (con password); la risposta della pagina è **sempre la stessa** ("se l'indirizzo esiste, abbiamo inviato..."), così non si rivela se un'email corrisponde a un account.

3. Modello dati: `fatt.UtenteToken`

3.1 Schema e migration

Migrations/013_UtenteToken.sql

```
CREATE TABLE fatt.UtenteToken
(
  Id          UNIQUEIDENTIFIER NOT NULL CONSTRAINT PK_UtenteToken PRIMARY KEY,
  UtenteId    UNIQUEIDENTIFIER NOT NULL,
  TokenHash   BINARY(32)       NOT NULL,    -- SHA-256 del token in chiaro (mai il token)
  Tipo        TINYINT          NOT NULL,    -- 0 = Attivazione, 1 = Reset
  ScadenzaUtc DATETIME2(0)     NOT NULL,
  UsatoUtc    DATETIME2(0)     NULL,        -- consumato (uso singolo)
  RevocatoUtc DATETIME2(0)     NULL,        -- sostituito da un reinvio
  CreatedAt   DATETIME2(3)     NOT NULL CONSTRAINT DF_UtenteToken_CreatedAt DEFAULT
(SYSDATETIME()),
  CONSTRAINT FK_UtenteToken_Utenti
    FOREIGN KEY (UtenteId) REFERENCES fatt.Utenti (IdUtente) ON DELETE CASCADE,
  CONSTRAINT UQ_UtenteToken_TokenHash UNIQUE (TokenHash)
);
CREATE INDEX IX_UtenteToken_UtenteId_Tipo ON fatt.UtenteToken (UtenteId, Tipo);
```

Scelte rilevanti: `BINARY(32)` per i 32 byte dell'hash SHA-256; **vincolo UNIQUE** sull'hash (la generazione è 256 bit casuali, collisione trascurabile); **ON DELETE CASCADE** (eliminando l'utente spariscono i suoi token); l'**indice** su `(UtenteId, Tipo)` per la revoca dei token attivi dello stesso tipo.

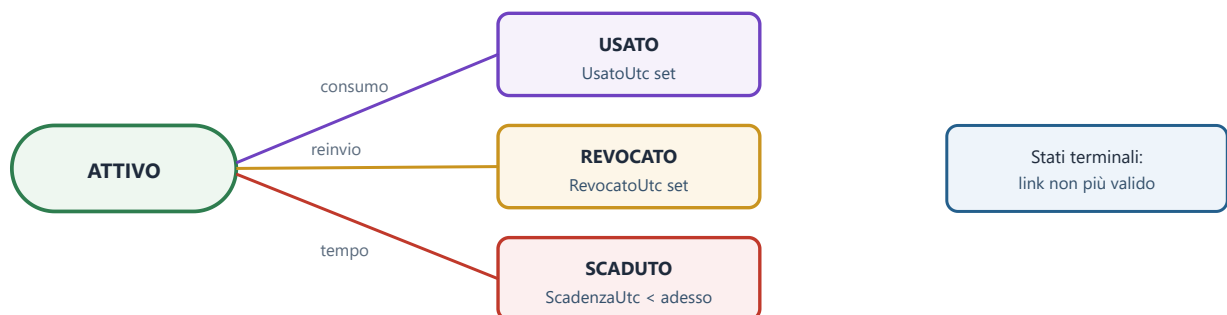
3.2 Stato derivato dalle colonne

Non esiste una colonna "Stato": lo stato è **derivato** da tre colonne nullable (pattern documentato in CLAUDE.md). Un token è **Attivo** se e solo se:

```
UsatoUtc IS NULL AND RevocatoUtc IS NULL AND ScadenzaUtc > (adesso)
```

Ogni transizione corrisponde a un evento datato che resta tracciato (utile in audit/diagnostica): `UsatoUtc` = quando è stato consumato, `RevocatoUtc` = quando un reinvio lo ha sostituito.

3.3 Macchina a stati del token



Le tre transizioni sono mutuamente irreversibili. La validazione le riconosce e produce un messaggio specifico per ciascuna (sezione 6).

4. Generazione del token e hashing

Il token in chiaro è generato con un RNG crittografico (32 byte) e codificato in **Base64Url** (url-safe, ~43 caratteri, senza padding): vive solo nell'URL dell'email. Nel database finisce esclusivamente il suo **SHA-256**. Alla validazione, il server ricalcola l'hash del token ricevuto e lo cerca: un dump della tabella è quindi inutile, perché non contiene link riutilizzabili.

Managers/UtenteTokenManager.cs – generazione e hashing

```
private static string GeneraToken()
{
    var bytes = RandomNumberGenerator.GetBytes(32);           // 256 bit casuali
    return Convert.ToBase64String(bytes)
        .TrimEnd('=').Replace('+', '-').Replace('/', '_');     // Base64Url, url-safe
}

private static byte[] CalcolaHash(string rawToken)
    => SHA256.HashData(Encoding.UTF8.GetBytes(rawToken));     // ciò che si salva
```

Hash, non cifratura

L'hash è a senso unico: dal valore in tabella non si risale al token. È la stessa logica con cui si custodiscono le password. Per questo il link va custodito dall'utente: il sistema non può "rimostrarlo".

La creazione del token usa `TimeProvider` per leggere "adesso", così le scadenze sono testabili senza attese reali:

Managers/UtenteTokenManager.cs – creazione (estratto)

```
private async Task<string> CreaAsync(Guid utenteId, UtenteTokenTipo tipo, int oreValidita,
CancellationTokens ct)
{
    var raw = GeneraToken();
    var nowUtc = _clock.GetUtcNow().UtcDateTime;
    var entity = new UtenteToken {
        Id = Guid.CreateVersion7(), UtenteId = utenteId, TokenHash = CalcolaHash(raw),
        Tipo = tipo, ScadenzaUtc = nowUtc.AddHours(oreValidita), CreatedAt = nowUtc,
    };
    await _repository.CreaRevocandoPrecedentiAsync(entity, ct); // revoca i precedenti + inserisce
    return raw;                                                  // il chiaro torna SOLO al chiamante
}
```

5. Repository: transazioni e sentinel TOCTOU

Il repository Dapper esegue le operazioni multi-statement in **transazione**. Due metodi sono critici.

Creazione con revoca dei precedenti (un reinvio invalida il vecchio link)

Repositories/UtenteTokenRepository.cs (estratto)

```
private const string SqlRevocaAttivi = ""
    UPDATE fatt.UtenteToken SET RevocatoUtc = SYSUTCDATETIME()
    WHERE UtenteId = @UtenteId AND Tipo = @Tipo
    AND UsatoUtc IS NULL AND RevocatoUtc IS NULL;""; // solo gli ancora-attivi

public async Task CreaRevocandoPrecedentiAsync(UtenteToken nuovo, CancellationToken ct)
{
    using var conn = await _connectionFactory.CreateOpenConnectionAsync(ct);
    using var tx = conn.BeginTransaction();
    try {
        await conn.ExecuteAsync(new CommandDefinition(SqlRevocaAttivi,
            new { nuovo.UtenteId, Tipo = (byte)nuovo.Tipo }, transaction: tx, cancellationToken: ct));
        await conn.ExecuteAsync(new CommandDefinition(SqlInsert, /* ...campi del nuovo token... */));
        tx.Commit();
    } catch { tx.Rollback(); throw; }
}
```

Consumo con sentinel (uso singolo a prova di race)

La `UPDATE` di "marca usato" porta nella clausola `WHERE` le stesse condizioni dello stato "Attivo": se nel frattempo il token è stato consumato/revocato/scaduto, aggiorna **zero** righe e la password non viene toccata.

Repositories/UtenteTokenRepository.cs (estratto)

```
private const string SqlMarkUsato = ""
    UPDATE fatt.UtenteToken SET UsatoUtc = SYSUTCDATETIME()
    WHERE Id = @Id AND UsatoUtc IS NULL AND RevocatoUtc IS NULL
    AND ScadenzaUtc > SYSUTCDATETIME();""; // sentinel: vincola lo stato

public async Task<int> ConsumaEImpostaPasswordAsync(Guid tokenId, Guid utenteId, string passwordHash,
    CancellationToken ct)
{
    using var conn = await _connectionFactory.CreateOpenConnectionAsync(ct);
    using var tx = conn.BeginTransaction();
    try {
        var righe = await conn.ExecuteAsync(new CommandDefinition(SqlMarkUsato,
            new { Id = tokenId }, transaction: tx, cancellationToken: ct));
        if (righe == 0) { tx.Rollback(); return 0; } // stato cambiato: niente password
        await conn.ExecuteAsync(new CommandDefinition(SqlSetPassword,
            new { UtenteId = utenteId, PasswordHash = passwordHash }, transaction: tx,
            cancellationToken: ct));
        tx.Commit(); return righe;
    } catch { tx.Rollback(); throw; }
}
```

Nota tecnica: la connection factory del progetto espone `IDbConnection`, quindi si usa `BeginTransaction()` sincrono passato a Dapper, coerentemente con gli altri repository.

6. Manager: validazione, ordine dei motivi, consumo atomico

Il manager espone `CreaAttivazioneAsync` / `CreaResetAsync` (emissione), `ValidaAsync` (lettura) e `ConsumaAsync` (uso). La validazione distingue **quattro motivi** tramite l'eccezione tipizzata `UtenteTokenInvalidoException` e li valuta in un **ordine pensato per la UX**: la revoca prevale sui motivi temporali, perché "link sostituito, usa l'ultimo" guida meglio dell'utente di "link scaduto".

Managers/UtenteTokenManager.cs – ValidaAsync

```
public async Task<UtenteToken> ValidaAsync(string rawToken, UtenteTokenTipo tipoAtteso,
CancellationToken ct)
{
    var entity = string.IsNullOrWhiteSpace(rawToken) ? null
        : await _repository.GetByHashAsync(CalcolaHash(rawToken), ct);

    // tipo sbagliato == "non trovato" per l'utente (indistinguibile)
    if (entity is null || entity.Tipo != tipoAtteso)
        throw new UtenteTokenInvalidoException(UtenteTokenInvalidoMotivo.NonTrovato, "...");
    if (entity.RevocatoUtc is not null) // 1) revoca esplicita
        throw new UtenteTokenInvalidoException(UtenteTokenInvalidoMotivo.Revocato, "...");
    if (entity.UsatoUtc is not null) // 2) già usato
        throw new UtenteTokenInvalidoException(UtenteTokenInvalidoMotivo.GiaUsato, "...");
    if (entity.ScadenzaUtc <= _clock.GetUtcNow().UtcDateTime) // 3) scaduto
        throw new UtenteTokenInvalidoException(UtenteTokenInvalidoMotivo.Scaduto, "...");
    return entity;
}
```

Motivo	Messaggio mostrato all'utente
NonTrovato	Link non valido. Verifica di averlo copiato per intero o richiedine uno nuovo.
Revocato	Questo link è stato sostituito da uno più recente. Usa l'ultimo ricevuto.
GiaUsato	Questo link è già stato utilizzato. Richiedine uno nuovo.
Scaduto	Questo link è scaduto. Richiedine uno nuovo.

I messaggi sono centralizzati in `UtenteTokenMessaggi`, usati sia dalle pagine sia dagli endpoint.

7. La doppia difesa contro le race condition

Tra il momento in cui la pagina valida il token ("check") e il momento in cui lo consuma ("use") lo stato potrebbe cambiare (un secondo click, un reinvio concorrente). Per questo la difesa è in **due punti**: il manager fa un pre-check che dà il messaggio giusto, ma la correttezza è garantita dal **sentinel** nel repository.

Managers/UtenteTokenManager.cs – ConsumaAsync

```
public async Task<Guid> ConsumaAsync(string rawToken, UtenteTokenTipo tipoAtteso, string passwordHash,
CancellationTokens ct)
{
    var entity = await ValidaAsync(rawToken, tipoAtteso, ct); // (A) pre-check: messaggio user-
friendly
    var righe = await _repository.ConsumaEImpostaPasswordAsync(entity.Id, entity.UtenteId,
passwordHash, ct);
    if (righe == 0) // (B) sentinel: stato cambiato fra
check e use
        throw new UtenteTokenInvalidoException(UtenteTokenInvalidoMotivo.GiaUsato, "...");
    return entity.UtenteId;
}
```



Perché entrambi

Il pre-check serve l'esperienza (spiega *perché* il link non va). Il sentinel serve la correttezza: anche se il manager fosse aggirato o lo stato cambiasse nell'istante sbagliato, l' **UPDATE** non tocca righe e la password non viene impostata. Un solo click "vince", gli altri falliscono in sicurezza.

8. Policy della password condivisa

La validazione della robustezza è centralizzata e applicata in **ogni** punto che imposta una password (set-password via link, creazione con password — usata solo dal seed — reset). Min 10 caratteri, con maiuscola, minuscola e cifra.

Authentication/PasswordPolicy.cs

```
public static string? Valida(string? password) // null = valida; altrimenti il messaggio d'errore
{
    if (string.IsNullOrEmpty(password) || password.Length < LunghezzaMinima) // 10
        return $"La password deve avere almeno {LunghezzaMinima} caratteri.";
    if (!password.Any(char.IsUpper)) return "La password deve contenere almeno una lettera maiuscola.";
    if (!password.Any(char.IsLower)) return "La password deve contenere almeno una lettera minuscola.";
    if (!password.Any(char.IsDigit)) return "La password deve contenere almeno una cifra.";
    return null;
}
```

Effetto collaterale gestito

Avendo innalzato la policy, la creazione di un utente *con* password e il seed dell'Admin richiedono ora una password conforme. Gli utenti già esistenti continuano a fare login: la verifica delle credenziali non riapplica la policy, controlla solo l'hash.

9. Invio email: `ISmailSender`, SMTP e modalità sviluppo

L'astrazione `ISmailSender` ha due implementazioni; quale usare si decide all'avvio in base alla configurazione: se `Smtp:Host` è valorizzato si invia davvero (MailKit), altrimenti il `LogEmailSender` scrive l'email — link compreso — nel log, così il flusso è provabile in locale senza un server SMTP.

Program.cs – selezione dell'implementazione

```
var smtpOptions = builder.Configuration.GetSection(SmtpOptions.SectionName).Get<SmtpOptions>();
if (smtpOptions?.IsConfigured == true)                // Smtp:Host non vuoto
    builder.Services.AddScoped<ISmailSender, SmtpEmailSender>();    // invio reale (MailKit)
else
    builder.Services.AddScoped<ISmailSender, LogEmailSender>();    // dev: link nel log
```

Email/SmtpEmailSender.cs (estratto) – invio via MailKit

```
var message = new MimeMessage();
message.From.Add(new MailboxAddress(_options.FromName, _options.FromAddress));
message.To.Add(MailboxAddress.Parse(toAddress));
message.Subject = subject;
message.Body = new BodyBuilder { HtmlBody = htmlBody }.ToMessageBody();

using var client = new SmtpClient();
var socketOptions = _options.UseStartTls ? SecureSocketOptions.StartTls : SecureSocketOptions.Auto;
await client.ConnectAsync(_options.Host, _options.Port, socketOptions, ct);
if (!string.IsNullOrEmpty(_options.User))
    await client.AuthenticateAsync(_options.User, _options.Password ?? string.Empty, ct);
await client.SendAsync(message, ct);
await client.DisconnectAsync(quit: true, ct);
```

I corpi delle email (oggetto + HTML) sono composti da `AccountEmailTemplates`, autoportanti (nessun asset esterno), con il link mostrato anche come testo per i client che non rendono i bottoni, e i valori interpolati sono sempre HTML-escaped.

L'URL assoluto del link è costruito in modo diverso a seconda del contesto: nell'invio admin-side dal `NavigationManager.BaseUri`; nell'endpoint forgot-password da `Request.Scheme + Request.Host`.

10. Endpoint di autenticazione e pagine

L'impostazione della password richiede `HttpContext` per firmare il cookie: per questo le pagine sono **static-rendered** e il loro form fa POST a **endpoint minimal-API**. In Blazor Server interattivo non si potrebbe chiamare `SignInAsync` da un componente.

10.1 Gli endpoint minimal-API

Program.cs – /auth/attiva (reset è identico col tipo Reset)

```
app.MapPost("/auth/attiva", async Task<IResult> (
    [FromForm] string Token, [FromForm] string Password, [FromForm] string PasswordConfirm,
    HttpContext httpContext, IUserTokenManager tokenManager, IUserManager utenteManager,
    IRuoloManager ruoloManager, IPasswordHasherService hasher, CancellationToken ct) =>
{
    var erroreInput = ValidaNuovaPassword(Password, PasswordConfirm); // policy + conferma
    if (erroreInput is not null) return Results.Redirect(BuildTokenRedirect("/attiva", Token,
        erroreInput));
    try {
        var hash = hasher.HashPassword(Password);
        var utenteId = await tokenManager.ConsumaAsync(Token, UtenteTokenTipo.Attivazione, hash, ct);
        var utente = await utenteManager.GetByIdAsync(utenteId, ct);
        if (utente is null) return Results.Redirect(BuildTokenRedirect("/attiva", Token, "Utente non
        trovato."));
        await SignInUtenteAsync(httpContext, utente, ruoloManager, ct); // sign-in automatico
        return Results.Redirect("/");
    } catch (UtenteTokenInvalidoException ex) {
        return Results.Redirect(BuildTokenRedirect("/attiva", Token,
        UtenteTokenMessaggi.Messaggio(ex.Motivo)));
    }
}).AllowAnonymous();
```

Tre funzioni locali sono condivise: `SignInUtenteAsync` (la stessa del login, vedi manuale permessi), `ValidaNuovaPassword` (policy + coincidenza dei due campi) e `BuildTokenRedirect` (rimanda alla pagina con l'errore in querystring).

10.2 Le pagine static-rendered

Le pagine `/attiva/{Token}`, `/reset-password/{Token}` e `/forgot-password` sono `[AllowAnonymous]`, condividono il guscio grafico `AuthShell` (stesso stile del login) e, per i magic-link, **validano il token in lettura** per decidere se mostrare il form o un messaggio d'errore. Il consumo effettivo avviene solo al POST.

Components/Pages/Account/Attiva.razor (estratto code-behind)

```
protected override async Task OnInitializedAsync()
{
    try { await TokenManager.ValidaAsync(Token, UtenteTokenTipo.Attivazione); _tokenValido = true; }
    catch (UtenteTokenInvalidoException ex) { _errore = UtenteTokenMessaggi.Messaggio(ex.Motivo); }
}
```

Le pagine anonime e la guardia di rotta

Perché queste pagine fossero raggiungibili senza login non basta `[AllowAnonymous]` : vanno anche inserite tra le "pagine di sistema" del `MenuService` (`"Attiva"` , `"ResetPassword"` , `"ForgotPassword"`), altrimenti la guardia di rotta dinamica le manda su `/access-denied` . È una regressione realmente incontrata e corretta.

10.3 Rate limiting e risposta neutra

Program.cs – forgot-password

```
app.MapPost("/auth/forgot-password", async Task<IResult> ( [FromForm] string Email, /* ... */ ) =>
{
    if (!string.IsNullOrEmpty(Email)) {
        var utente = await utenteManager.GetByEmailAsync(Email, ct);
        if (utente is { Attivo: true, Email: not null } && !string.IsNullOrEmpty(utente.PasswordHash))
        {
            var token = await tokenManager.CreaResetAsync(utente.IdUtente, ct);
            var link = $"{HttpContext.Request.Scheme}://{HttpContext.Request.Host}/reset-
password/{token}";
            var (subject, body) = AccountEmailTemplates.Reset(link, utente.Username,
tokenOptions.Value.ResetOreDefault);
            await emailSender.SendAsync(utente.Email, subject, body, ct);
        }
    }
    return Results.Redirect("/forgot-password?inviato=1"); // esito SEMPRE uguale (no enumeration)
}).AllowAnonymous().RequireRateLimiting("forgot-password"); // max 5 richieste/min per IP
```

11. Allineamento della gestione utenti

Per chiudere il cerchio, la creazione utente è stata allineata al modello di ICMVerbali: l'admin non può mai impostare una password.

- Nel dialog "Nuovo utente": **email obbligatoria** (è il canale del link), **nessun campo password**, avviso informativo. L'utente nasce sempre "invitato".
- Alla creazione, il **link di attivazione parte automaticamente**.
- **Rimossa** l'impostazione manuale della password (eliminato `ImpostaPasswordDialog` e il relativo pulsante). Resta l'azione "Invia link" (attivazione se da-attivare, reset se già attivo).
- Il metodo di dominio `ImpostaPasswordAsync` resta nel manager (testato) ma non è più esposto in UI: l'impostazione passa sempre dal circuito email.

Components/Shared/UtenteFormDialog.razor – creazione (estratto)

```
// Email obbligatoria solo in creazione:
< MudTextField @bind-Value="_model.Email" Required="@(!IsModifica)" RequiredError="Email obbligatoria"
... />
// Crea SEMPRE invitato (password null) e ritorna l'IdUtente per l'invio automatico del link:
var idNuovo = await UtenteManager.CreaAsync(_model.Username, password: null, _model.Email,
_model.IdRuolo);
MudDialog.Close(DialogResult.Ok<Guid?>(idNuovo));
```

12. Configurazione e segreti (Brevo)

La sezione `Smtp` di `appsettings.json` contiene i parametri non sensibili; la **SMTP key va in user-secrets** (mai versionata). Con `Smtp:Host` vuoto si resta in modalità sviluppo (link nel log).

`appsettings.json` (estratto)

```
"UtenteToken": { "AttivazioneOreDefault": 168, "ResetOreDefault": 1 },
"Smtp": { "Host": "smtp-relay.brevo.com", "Port": 587, "User": "...@smtp-brevo.com",
          "FromAddress": "job@lucaschiavon.eu", "FromName": "ICM Fatturazioni", "UseStartTls": true }
```

```
# la chiave segreta NON va in appsettings:
dotnet user-secrets set 'Smtp:Password' 'LA-SMTP-KEY' --project src/ICMFatturazioni.Web
```

Nota operativa: in PowerShell usare gli **apici singoli** per la key, così caratteri come `$` non vengono interpretati. Un errore **535 5.7.8 Authentication failed** indica credenziali SMTP errate (key non valida, o login non corrispondente).

Il mittente (`FromAddress`) deve essere un **sender verificato** sul provider, altrimenti l'invio viene rifiutato.

13. Elenco esaustivo dei punti di codice toccati

Legenda: **NUOVO** file creato · **MOD** file modificato · **RIMOSSO**.

File	Tipo	Ruolo nel flusso
Migrations/013_UtenteToken.sql	NUOVO	Tabella token (hash, tipo, scadenza, FK cascade)
Entities/UtenteToken.cs	NUOVO	POCO del token; stato derivato dalle colonne
Entities/UtenteTokenTipo.cs	NUOVO	Enum Attivazione/Reset (persistito tinyint)
Managers/UtenteTokenInvalidoException.cs	NUOVO	Eccezione tipizzata + enum motivi (ordine UX)
Repositories/Interfaces/IUtenteTokenRepository.cs	NUOVO	Contratto: crea-revocando, get-by-hash, consuma
Repositories/UtenteTokenRepository.cs	NUOVO	Dapper: transazioni + sentinel TOCTOU
Managers/Interfaces/IUtenteTokenManager.cs	NUOVO	Contratto: crea/valida/consuma
Managers/UtenteTokenManager.cs	NUOVO	Generazione, hashing, validazione, consumo (TimeProvider)
Authentication/UtenteTokenOptions.cs	NUOVO	Durate token (168 h / 1 h)
Authentication/UtenteTokenMessaggi.cs	NUOVO	Messaggi/titoli user-friendly per motivo
Authentication/PasswordPolicy.cs	NUOVO	Policy robustezza condivisa (min 10 + complessità)
Email/IEmailSender.cs	NUOVO	Astrazione di invio email
Email/SmtpEmailSender.cs	NUOVO	Invio reale via MailKit (STARTTLS)
Email/LogEmailSender.cs	NUOVO	Dev: scrive l'email (link) nel log
Email/SmtpOptions.cs	NUOVO	Config SMTP + flag <code>IsConfigured</code>
Email/AccountEmailTemplates.cs	NUOVO	Corpi HTML invito/reset (escape, link testuale)
Components/Account/AuthShell.razor	NUOVO	Guscio grafico condiviso delle pagine account
Components/Pages/Account/Attiva.razor	NUOVO	Pagina <code>/attiva/{token}</code> (imposta password)
Components/Pages/Account/ResetPassword.razor	NUOVO	Pagina <code>/reset-password/{token}</code>
Components/Pages/Account/ForgotPassword.razor	NUOVO	Pagina <code>/forgot-password</code> (risposta neutra)
Program.cs	MOD	DI (token, email, TimeProvider, rate limiter), 3 endpoint <code>/auth/*</code> , helper <code>SignInUtenteAsync</code> condiviso col login

Managers/UtenteManager.cs	MOD	Policy in CreaAsync / ImpostaPasswordAsync ; GetByEmailAsync
Managers/Interfaces/IUtenteManager.cs	MOD	GetByEmailAsync ; nota su ImpostaPasswordAsync non-UI
Repositories/UtenteRepository.cs · IUtenteRepository.cs	MOD	GetByEmailAsync (lookup per email)
Managers/MenuService.cs	MOD	Pagine account aggiunte a PagineSistema
Components/Shared/UtenteFormDialog.razor	MOD	Email obbligatoria, niente password, crea invitato
Components/Pages/Admin/GestioneUtenti.razor	MOD	"Invia link", auto-invio post-creazione; rimosso reset manuale
Components/Pages/Login.razor	MOD	Link "Password dimenticata?"
appsettings.json	MOD	Sezioni Smtip e UtenteToken
ICMFatturazioni.Web.csproj	MOD	PackageReference MailKit 4.16.0
Components/Shared/ImpostaPasswordDialog.razor	RIMOSSO	Impostazione manuale password (non più ammessa)

Test aggiunti/aggiornati

File	Tipo	Copertura
Managers/UtenteTokenManagerTests.cs	NUOVO	Generazione/hash, ordine motivi, scadenza, monouso (TimeProvider mutabile)
Managers/FakeUtenteTokenRepository.cs	NUOVO	Doppio in-memory con sentinel replicato
UtenteManagerTests.cs · DatabaseSeederTests.cs	MOD	Password conformi alla nuova policy
MenuServiceTests.cs	MOD	Pagine account fra le pagine di sistema

14. Verifica e test

La logica del manager è coperta da unit test (xUnit, fake in-memory, niente DB), con un `TimeProvider` pilotabile per provare scadenze e revoche senza attese reali. I casi verificati includono: generazione che salva solo l'hash; scadenza corretta; revoca del token precedente a ogni reinvio; ordine dei motivi (revocato prevale su scaduto); consumo monouso; consumo di un token scaduto che non imposta la password.

Verifica	Esito
Build soluzione	0 warning, 0 errori
Suite unit test	113 test verdi
Smoke end-to-end (forgot → email → reset → sign-in → monouso)	PASS
Invio reale via Brevo	PASS (con SMTP key valida in user-secrets)
Errori a runtime (<code>fatt.LogErrors</code>)	0

In una frase

Un token monouso a tempo, di cui il DB conserva solo l'hash, consumato in modo atomico con doppia difesa, consente all'utente di impostare da sé una password robusta: l'amministratore invita e (re)invia il link, ma non conosce mai la password.